

Задание

Разработать программу для импорта банковской выписки

Программа получает файл операций по счету, отделяет корректные операции от ошибочных строк и формирует отчет об импорте

Программа должна получать пути к файлам выписок через аргументы командной строки

Пример запуска:

```
./bank_import valid_statement.txt mixed_statement.txt  
edge_cases_statement.txt
```

Для проверки использовать следующие входные файлы:

Файл	Содержимое
valid_statement.txt	корректная выписка
mixed_statement.txt	выписка с корректными и ошибочными строками
edge_cases_statement.txt	выписка с граничными случаями
missing_statement.txt	имя отсутствующего файла для проверки ошибки открытия

Файл содержит одну банковскую операцию на строку:

```
id;date;type;amount;description
```

Например:

```
TX1001;2026-09-01;TRANSFER;-1250.00;Taxi  
TX1002;2026-09-01;PAYMENT;-249.90;Coffee  
TX1003;2026-09-02;INCOME;75000.00;Salary  
TX1004;2026-09-03;FEE;-99.00;
```

Поля имеют следующий смысл:

Поле	Значение
id	уникальный идентификатор операции
date	дата в формате YYYY-MM-DD
type	тип операции
amount	сумма операции
description	описание операции

Использовать следующие типы операций:

Тип	Правило суммы
INCOME	сумма больше 0
PAYMENT	сумма меньше 0
TRANSFER	сумма не равна 0
FEE	сумма меньше 0

Поле `description` является необязательным. Пустое описание должно храниться как отсутствие значения

В данных могут встречаться ошибки:

- пустая строка
- неверное количество полей
- пустой `id`
- дублирующийся `id`
- дата в неверном формате
- неизвестный тип операции
- сумма в неверном формате
- сумма, которая нарушает правило типа операции
- невозможность открыть файл
- запуск программы без аргументов командной строки

Ошибки отдельных строк должны попадать в отчет и не должны останавливать импорт всего файла. Корректные строки должны быть импортированы

Запуск без аргументов командной строки должен обрабатываться как ошибка использования программы. Программа должна вывести понятное сообщение с примером запуска

Невозможность открыть файл должна обрабатываться как исключительная ситуация. Программа должна перехватить исключение, вывести понятное сообщение и продолжить обработку остальных файлов, если они есть

Для проверки внутренних предположений программы использовать `assert`. Например, после успешного `parsing` операции тип операции должен быть одним из допустимых значений

Для описания операции использовать отдельный тип, который хранит отсутствие описания через `std::optional`

В программе должен быть объект, который открывает файл и закрывает его автоматически при выходе из `score`

Программа должна обработать все файлы, переданные через аргументы командной строки. Для проверки ошибки открытия среди аргументов нужно передать `missing_statement.txt`

Для каждого обработанного файла вывести:

- имя файла
- количество прочитанных строк
- количество успешно импортированных операций
- количество ошибочных строк
- список ошибок с номером строки и причиной
- итоговый баланс по импортированным операциям
- сумму доходов
- сумму расходов

Интерактивный CLI реализовывать не требуется. Достаточно обработать аргументы командной строки

Требования

- Названия переменных, функций, типов и других сущностей программы должны быть осмысленными и записываться на английском языке без транслитерации
- Программа должна читать пути к входным файлам из аргументов командной строки
- Программа должна обрабатывать запуск без аргументов командной строки
- Ошибки отдельных строк должны накапливаться в отчете об импорте
- Невозможность открыть файл должна обрабатываться через `throw`, `try` и `catch`
- В программе должен использоваться тип исключения, связанный с `std::exception`
- В программе должен быть хотя бы один осмысленный `assert` для проверки внутреннего состояния
- Отсутствующее описание операции должно храниться через `std::optional`
- Файловый поток должен использоваться как RAII-объект
- Программа должна продолжать импорт файла после ошибки в отдельной строке
- Программа должна продолжать обработку следующих файлов после ошибки открытия одного файла
- Программа должна быть грамотно разбита на модули и содержать как минимум один `.hpp` файл и два `.cpp` файла. Допускается использовать больше файлов
- Для заголовочных файлов использовать `#pragma once`
- Программа должна компилироваться и выполняться без ошибок
- Студент должен понимать написанный код, уметь обосновать свое решение и отвечать на вопросы во время защиты лабораторной работы

Минимальный набор функций:

- `parseOperationType`
- `parseDate`
- `parseAmount`
- `parseTransaction`
- `parseCommandLine`
- `importStatement`
- `validateTransaction`
- `calculateBalance`
- `printImportReport`